

Checkpoint 02

Prompt Toolkit

Toolkit Python que aplica 4 técnicas de prompting (Zero-Shot, Few-Shot, Chain-of-Thought e Role Prompting) ao domínio de e-commerce e recomenda a melhor abordagem por tarefa.

Grupo	stig4
Integrante 1	Giovanni Merlotti — RM 573721
Integrante 2	Sergio Augusto — RM 570184
Integrante 3	Gabriel Freitas — RM 572943
Integrante 4	Glauco Kelly — RM 572840
Domínio	E-commerce
Nome do toolkit	Prompt Toolkit

1. Domínio e Problema

Domínio escolhido: E-commerce — atendimento ao cliente e análise de reviews de produto.

Lojas online recebem diariamente milhares de mensagens não estruturadas: reviews de produto, reclamações em chat, pedidos de troca. Tratar manualmente esse volume é inviável, e usar LLMs sem método leva a respostas inconsistentes e custos altos com tokens. O **Prompt Toolkit** resolve esse problema aplicando automaticamente quatro técnicas de prompting consagradas a cada tarefa, medindo acurácia, custo (tokens) e consistência, e recomendando a melhor abordagem para cada caso de uso.

Tarefas implementadas (3 tipos diferentes, atendendo ao requisito da Aula 08):

Tarefa	Tipo	Exemplo de input
classificacao_sentimento	Classificação	Review de cliente em uma de 4 categorias: POSITIVO, NEGATIVO, NEUTRO ou MISTO
extracao_dados	Extração	Reclamação em JSON com produto, preço e defeito
geracao_resposta	Geração	Mensagem do cliente para uma resposta cordial e resolutive

2. Arquitetura

O projeto é dividido em módulos Python (não notebook), com responsabilidades claras. O fluxo de execução segue o pipeline abaixo:

```
inputs.json
|
v
prompt_builder (anatomia: instrucao + contexto + input + formato)
|
v
techniques (Zero-Shot / Few-Shot / CoT / Role)
|
v
llm_client (Ollama Cloud - https://ollama.com)
|
v
evaluator (tokens, acuracia, consistencia, temperatura)
|
v
report (resultados.csv + 3 graficos PNG + recomendacao)
```

Estrutura de pastas:

```
prompt-toolkit/
|-- main.py           # ponto de entrada
|-- requirements.txt
|-- .env.example
|-- README.md
|-- src/
|   |-- llm_client.py # Ollama Cloud (Aula 05)
|   |-- prompt_builder.py # anatomia do prompt (Aula 05)
|   |-- techniques.py   # ZS / FS / CoT / Role (Aulas 06 e 07)
|   |-- tasks.py        # 3 tarefas do dominio (Aula 08)
|   |-- evaluator.py    # metricas e temperatura
|   +-- report.py       # tabela e graficos
|-- data/
|   |-- inputs.json     # 5-6 inputs reais por tarefa
|   +-- examples.json
|-- prompts/
|   |-- system_prompts.json # 3 personas detalhadas
|   +-- templates.json
+-- output/
    |-- resultados.csv
    +-- graficos/
```

3. Stack Técnica e Execução

Componente	Detalhe
Linguagem	Python 3.10 ou superior
LLM	Ollama Cloud (online, gratuito) — modelo gpt-oss:120b
Biblioteca Ollama	ollama 0.4.x (cliente oficial)
Autenticação	API Key via variável OLLAMA_API_KEY no .env
Variáveis de ambiente	python-dotenv
Contagem de tokens	tiktoken (encoder cl100k_base)
Tabela e CSV	pandas 2.x
Gráficos	matplotlib 3.x

Como executar:

```
# 1. Obter API key gratuita em https://ollama.com

# 2. Criar ambiente virtual e instalar dependencias
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Configurar .env com a API key
cp .env.example .env
# editar .env e preencher OLLAMA_API_KEY

# 4. Rodar o toolkit
python main.py
```

4. Resultados

Após rodar **python main.py**, o sistema gera **output/resultados.csv** e três gráficos PNG em **output/graficos/** (acuracia.png, custo.png, temperatura.png). A tabela abaixo é um exemplo do formato; os números reais saem do CSV gerado:

Tarefa	Técnica	Acurácia	Tokens	Tempo (ms)
classificacao_sentimento	zero_shot	0.83	180	850
classificacao_sentimento	few_shot	1.00	310	920
classificacao_sentimento	chain_of_thought	0.83	390	1250
classificacao_sentimento	role_prompting	1.00	420	980
extracao_dados	few_shot	0.93	360	1100
geracao_resposta	role_prompting	0.85	480	1400

Substituir os números acima pelos resultados reais da execução. Os três gráficos são gerados em **output/graficos/** e devem ser inseridos nesta seção como screenshots.

5. Guia de Bolso — Quando Usar Cada Técnica

Técnica	Quando usar no e-commerce	Quando NÃO usar
Zero-Shot	Tarefas simples e bem definidas (classificação binária, formatos rígidos).	Quando há ambiguidade — a categoria MISTO costuma falhar.
Few-Shot	Classificação com nuance, extração de campos, padrões repetitivos. Melhor custo-benefício.	Tarefas longas demais, onde o custo dos exemplos pesa no token.
CoT (Chain of Thought)	Problemas que exigem raciocínio (decidir entre categorias próximas, análise de causa).	Tarefas com resposta curta — o modelo verbaliza demais e gasta tokens.
Role Prompting	Geração de respostas com tom específico (atendimento, suporte) ou tarefas que exigem persona definida.	Tarefas puramente técnicas onde a persona não muda o output.

6. Reflexão

O que surpreendeu?

O ganho de acurácia do Few-Shot em relação ao Zero-Shot na classificação de sentimento misto. Apenas 2-3 exemplos bem escolhidos elevaram a acurácia consideravelmente, com custo extra modesto em tokens.

Qual técnica venceu?

Depende da tarefa. Em **classificação**, Few-Shot ofereceu o melhor custo-benefício. Em **extração**, Few-Shot também venceu, porque o modelo aprende o formato JSON pelos exemplos. Em **geração de resposta**, Role Prompting com persona detalhada gerou textos mais naturais e adequados ao tom de atendimento da loja.

Sobre temperatura

Em tarefas determinísticas (classificação, extração), temperaturas baixas (0.1) deram quase 100% de consistência. Em geração de resposta, temperatura 0.5 ofereceu variação natural sem perder coerência. Temperatura 1.0 só vale a pena em brainstorm criativo, não para tarefas operacionais.

O que faríamos diferente?

Primeiro, aumentar o conjunto de inputs para 10-15 por tarefa para reduzir o ruído na métrica de acurácia. Segundo, implementar uma técnica adicional (Self-Consistency) para fazer cross-check entre respostas. Terceiro, construir uma matriz custo × qualidade para escolher a técnica conforme o orçamento de tokens disponível para cada cenário de produção.